Cotton Boll Annotation Guide

Applied Vision Lab — Texas Tech University

This guide exists so that **everyone annotates the same way**. Inconsistent annotations are worse than no annotations, because the model learns the inconsistency.

---

1. Terminology — learn these words first

Term	What it means
Square	The flower bud stage, *before* the flower opens. **Not a boll. Do not annotate.**
Boll	The seed pod that forms after the flower drops. **This is what we annotate.**
Bracts	The green, leaf-like structures that wrap around the **base** of the boll. **Part of the boll — you annotate them.** NOT an occluder.
Non-bract portion	The exposed pod above the bracts. ⚠ **Used ONLY for the visibility check** (see below). You still annotate the whole boll.
Tip	The pointed top end of the boll.
Base	Where the boll meets the stem, below/behind the bracts.
Tag	A physical paper/plastic tag hung on a boll by the Plant & Soil team with a thin thread. **It carries the boll's real ID.**

We annotate CLOSED, GREEN bolls.

IMPORTANT — do not confuse these two things

> ### You annotate the WHOLE BOLL. Bracts included.
>
> Your `cotton_boll` polygon goes around the **entire** boll — the bracts **and** the pod above them. The bracts are part of the boll. **Never** draw a polygon that excludes the bracts.
>
> ---
>
> The term **"non-bract portion"** is used for **exactly one purpose**:
>
> **→ Deciding whether a boll is too occluded to annotate at all.**
>
> When something is covering a boll, you ask: "*is at least 50% of the **non-bract portion** still visible?*" If yes → annotate it (the whole boll, bracts included). If no → skip it.
>
> **That is the only place "non-bract portion" is ever used.** It does not change what you draw.

---

2. `boll_id` — THE MOST IMPORTANT RULE

`boll_id` = the number on the physical tag. Nothing else.



The Plant & Soil team hangs a tag on each boll with a thin thread. ***The number on that tag is the*** `boll_id`***.*** *Note the tag hangs* ***below*** *the boll — you often have to look down and away from the boll to read it.*

The Plant & Soil team has tagged bolls with real IDs and has **ground-truth size measurements** for each one. Our `boll_id` links our annotation to their measurement. If the ID is wrong, the link is wrong, and the data is silently corrupted.

NEVER invent, guess, or assign a random `boll_id`.

A wrong ID is worse than no annotation at all.

If you can't read the tag in the current frame:

1. **Scrub forward and backward** through the frames — the tag rotates and becomes readable.
2. **Check the other camera angles** — the same boll is filmed from front / back / right.
3. Only when you have actually read the number, annotate.

Yes, this is slow. It is the only correct way.

Same boll = same `boll_id`, always

Across every frame, and across every camera angle. Use **Track mode** for almost everything — it carries the ID forward automatically.

Untagged bolls (field data only)

All bolls in the **greenhouse** data are tagged. You should always be able to find an ID.

Field data may contain untagged bolls. If a boll genuinely has no tag:

- Assign it an ID from the **reserved untagged range:** `9000` **and above** (9000, 9001, 9002, …).
- **Never reuse an ID.** Two different bolls must never share an ID.
- Keep a note of which numbers you've used so the next person doesn't collide with them.

> **Why a reserved range?** Tag IDs link to the Plant & Soil team's ground-truth size measurements. If an untagged boll were given, say, ID 47, it would be indistinguishable from real tagged boll #47. Anything at 9000+ is known to have **no ground-truth measurement**, so it can still train the detector while being safely excluded from the sizing analysis.

---

3. Should I annotate this object at all?

Annotate if:

- You can tell it is a **boll** from **its own appearance** — its shape, size, and structure.

Do NOT annotate if:

- **You're only sure it's a boll because you see a tag on it.** The tag is a crutch. At inference time the model has no tags. If the visual evidence alone is ambiguous, the model can't learn anything useful from it.

- **You can't tell if it's a real boll or just bracts.** Late-stage squares and bract structures look similar to small bolls. **When in doubt, skip it.**
- It's a **square** (flower bud stage).
- It's an **open/white** boll.

> **Rule of thumb:** if you'd have to squint, argue with yourself, or check the tag to decide — skip it. A missing annotation costs us one training sample. A wrong annotation teaches the model something false.

An example of boll to skip because we know its a boll only because of the tag.



---

4. THE DECISION TREE

Work through this for **every** boll, in order.

STEP 1 — Is any part of the boll hidden?

Look at the **non-bract portion** (the exposed pod above the bracts) — this is the *visibility test only*.

- Bracts are **part of the boll**. A boll wrapped in its own bracts is **NOT** occluded.
- ⚠ Reminder: whatever you decide here, the polygon you draw always covers the **whole boll, bracts included.**

```
Is anything (leaf / stem / another boll / a tag from a different boll)
covering part of the non-bract portion?

├── NO, it's fully visible
│      → ONE polygon
│        annotation_type = whole_visible
│        occluded_by     = none
│
├── YES, it's cut off by the IMAGE BORDER **and nothing else**
│      → ⭐ SPECIAL CASE — see Step 3
│      (If a leaf/stem/other boll ALSO covers it → the real occluder wins. Go to Step 2.)
│
└── YES, something is covering it
       → Is at least 50% of the non-bract portion still visible?
            ├── NO  → skip this boll in this frame (too hidden to be useful)
            └── YES → go to Step 2 (occluded boll — TWO polygons)
```

STEP 2 — Occluded boll → you need TWO kinds of polygon

An occluded boll gets **BOTH**:

A. `whole_estimate` — your best outline of the **complete** boll (bracts **and** pod), *including the parts hidden by the occluder.* **B.** `visible_parts` — polygon(s) covering **only** what you can actually **see** of the boll (again: bracts count as boll). If the occluder splits the boll into two visible chunks, draw **two** `visible_parts` polygons.

> **NEVER use `whole_visible` on an occluded boll.**



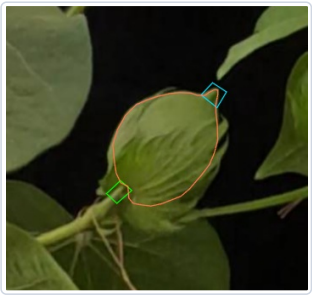
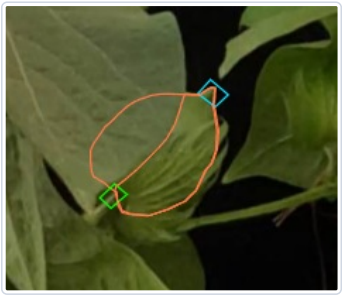
`whole_estimate` *(pink) = the full boll including the hidden part.* `visible_parts` *(red) = only what you can actually see.* ***Both*** *are drawn. → More examples in §6.5–6.7.*

How to get `whole_estimate` right — SCRUB FIRST

Before you guess the hidden shape:

> **Scrub forward and backward through the frames. Is this same boll ever fully visible, with no occlusion, in ANY frame?**
>
> - **YES** → Go to that frame. Look at its real size and shape. **Use that** as your `whole_estimate` in the occluded frames. This is far more accurate than guessing.
> - **NO** (occluded in every frame) → Only then, make your best guess for the hidden part.

This is the single biggest accuracy win in the whole workflow. Do not skip it.

	
Don't guess the hidden shape here…	**…scrub and find its REAL size here.**

→ Full walkthrough in §6.10.

Set `occluded_by`

Value	Use when
`leaf`	A leaf covers part of it
`stem`	A stem covers part of it
`other_boll`	Another boll covers part of it
`frame_edge`	Cut off by the image border (see Step 3)
`other`	**Anything else** — e.g. a tag belonging to a *different* boll drifting in front of this one
`none`	Not occluded

> A boll's **own** tag hangs on a thin thread and is not treated as an occluder — ignore it. A tag from a **different** boll that covers this one → `occluded_by = other`.

What if there's more than one occluder?

`occluded_by` takes only ONE value. Two rules:

1. Real occluder beats `frame_edge`. If a boll is cut off by the image border **AND** partly behind a leaf/stem/other boll: → treat it as a normal **occluded** boll (`whole_estimate` + `visible_parts`), and set `occluded_by` to the **real occluder** (leaf / stem / other_boll / other). The frame_edge exception only applies when the border is the **only** thing cutting it off.

2. Two real occluders → pick the one that covers MORE of the boll. (This is rare. If two things are covering a boll, it usually fails the ≥50% visibility rule anyway and you should skip it.)

STEP 3 — SPECIAL CASE: cut off by the image border (and NOTHING else)

The boll runs off the edge of the frame. It is not hidden by an object — it's just outside the picture.

> ⚠ **Only** use this if the border is the **only** thing cutting it off. If a leaf, stem, or another boll is **also** covering it, that's a normal occluded boll — go back to Step 2 and use `whole_estimate` + `visible_parts`, with `occluded_by` set to the real occluder.

```
annotation_type = whole_visible     ← YES, whole_visible. This is the exception.
occluded_by     = frame_edge
```



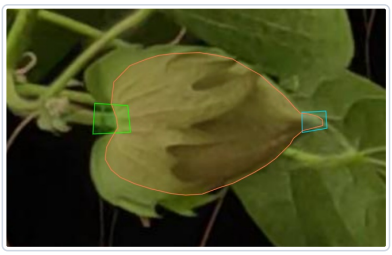
Why? We do not estimate geometry that is off-screen. The part inside the frame IS the annotation. Draw one polygon over the visible portion.

→ *See §6.8 for the "too far out of frame → skip" case.*

The same **≥50% of the non-bract portion visible** rule still applies — if barely any of it is in frame, skip it.

STEP 4 — TIP

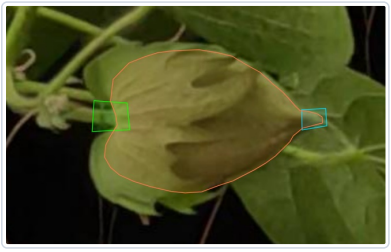
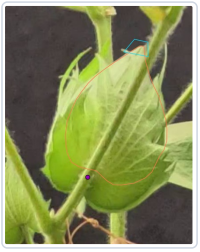
```
Can you SEE the tip clearly?
├── YES  → draw a tip_polygob  (4-point QUAD — see Section 5)
│          tip_visibility = visible
│
├── NO, but you can reasonably infer where it is
│      (e.g. hidden behind a leaf, but the boll's shape tells you where the tip must be)
│      → place a tip POINT
│        tip_visibility = guessed
│
└── NO, and you cannot even guess
       (e.g. the boll is oriented so the tip points away from the camera,
        or it's completely hidden)
       → draw NOTHING
         tip_visibility = unknown
```

		
visible → QUAD	**guessed** → POINT	**unknown** → NOTHING

→ *Detail in §6.11–6.12.*

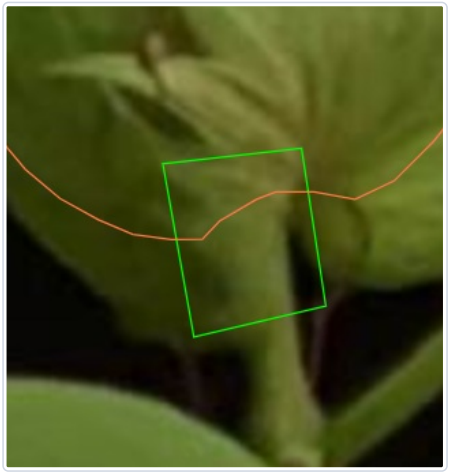
STEP 5 — BASE

Exactly the same logic, using `base_rectangle` (quad) / base point / nothing, and `base_visibility` = visible / guessed / unknown.

		
visible → QUAD	**guessed** → POINT	**unknown** → NOTHING

→ *Detail in §6.13.*

THE GOLDEN RULE — visibility flag must MATCH what you drew

Flag	What must be drawn	Looks like
`visible`	A **quad polygon** (`tip_polygob` / `base_rectangle`)	
`guessed`	A **point** (`tip` / `base`)	
`unknown`	**Nothing**	

This is the rule our validator checks. Before any annotation task is accepted, we run `validate_annotations.py` over the export — it flags every case where the flag and the drawing disagree, so the dataset stays consistent across everyone working on it.

Every time you change what you drew, check the flag. Every time you change the flag, check the drawing.

4.5 HOW TO DRAW THE BOLL POLYGON — the bract problem

This is subtle and it matters a lot. Read it carefully.

The problem

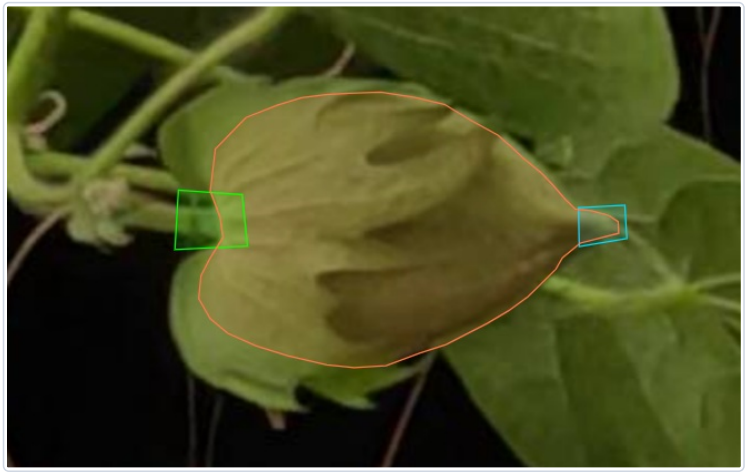
Bracts flare out. They splay away from the boll body, they curl, they're wispy, and they can extend a long way past the actual boll. If one person traces every bract tip and another keeps the outline close to the boll body, the two masks look completely different — even though both people annotated "the same boll."

The rule

> **Keep the polygon confined to the actual boll shape.**
>
> Follow the boll's real body. **Do not chase bracts that flare far out from it.**

- Include the bracts that **hug** the boll and form part of its body.
- **Do not** extend the polygon out to catch every splayed, curled, or stray bract tip.
- The outline should read as **the shape of the boll** — not a spiky star following every bract.
- Aim for a **consistent, plausible boll outline**, so that the same boll drawn by two different people looks the same.

What it looks like in practice



Look at the **orange polygon**. Notice:

- The bracts extend **well past** the orange line — up-left, down-left, and to the right. **The polygon does not follow them.**
- The polygon traces the **boll's actual body** — a clean, closed, boll-shaped outline.
- This is what "confined and consistent with the actual boll" means.

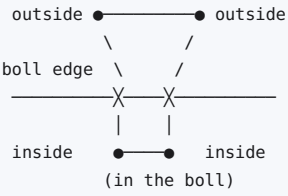
> **Test yourself:** if you removed the image and just looked at your polygon, would it still read as the outline of a cotton boll? If it looks like a spiky star or a leaf, you've chased the bracts too far.

---

5. The tip/base QUAD rule — 2 IN, 2 OUT

`tip_polygob` and `base_rectangle` are **exactly 4 points**. Not 3. Not 5.

Draw them with roughly 2 vertices INSIDE the boll and 2 vertices OUTSIDE.



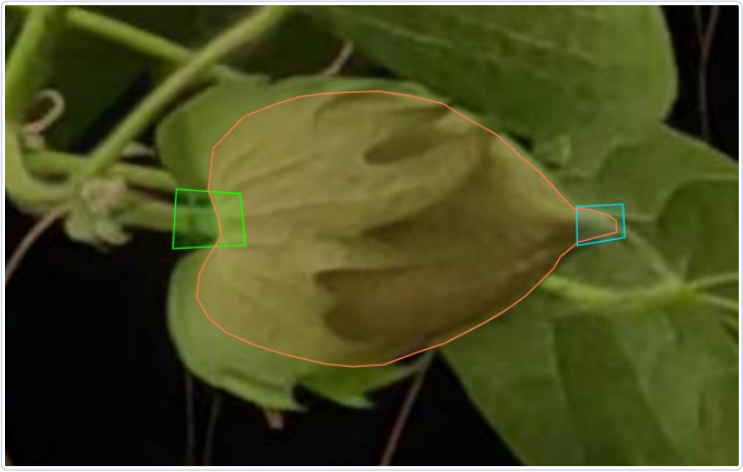
Why? (You need to understand this, or you'll draw them wrong.)

The model does not learn "the tip is at pixel (x, y)." It learns a **heatmap** — a blurry blob of probability over the region. To learn *where the tip is*, it needs to see both:

- what the **tip** looks like (inside), and
- what the **background right next to the tip** looks like (outside).

If you draw the quad entirely inside the boll, the model never sees the boundary and learns a much weaker signal.

See it on a real annotation



- **Green quad** and **cyan quad** — each one **straddles the orange boll outline**.
- Roughly **half of each quad is inside the boll, half is outside**. That's the 2-in / 2-out rule.
- Each quad is **small compared to the whole boll** — but it clearly contains **background** as well as the boll edge.

The quad is a REGION, not an outline

You are **not** tracing the exact shape of the tip. You are marking a quad **centered on** the tip that **straddles the boll's edge**.

Sizing it:

- It should be **very small compared to the whole boll**.
- But it **must include some background** along with the tip — that is the entire point of the 2-in / 2-out rule. If it hugs the tip too tightly, there's no background inside it and the model loses the signal.
- Don't go too large either — a quad covering a big chunk of the boll is too coarse to be useful.

> **Small relative to the boll, but big enough that background is clearly inside it.**

Common mistakes

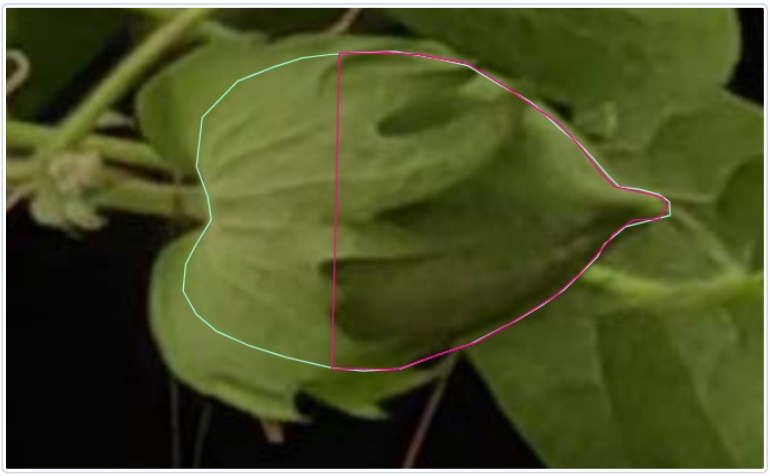
Wrong	Why
All 4 points inside the boll	**No background in the quad** — the model loses the signal
Quad hugging the tip too tightly	Same problem — barely any background gets included
All 4 points outside the boll	Doesn't mark the tip at all
Quad as big as the whole boll	Too coarse — the heatmap becomes meaningless
3 points or 5 points	Schema violation — the validator will reject it

---

6. Hard cases — real examples from our dataset

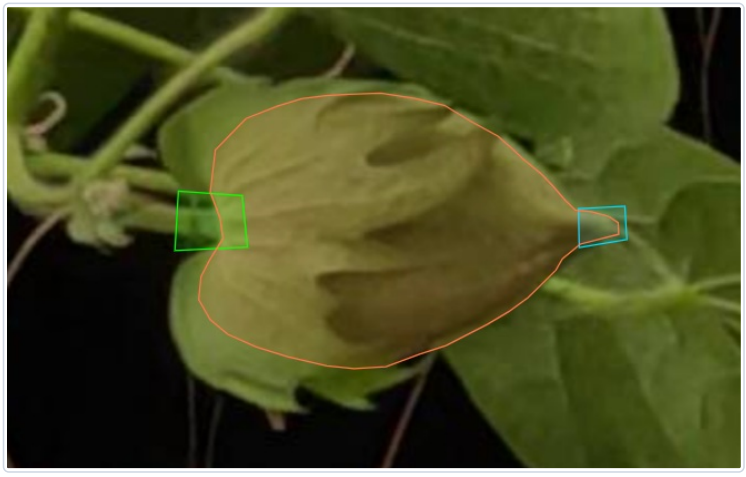
> **Every example below is a real frame from our own annotated data.**

6.1 — Anatomy: what you draw vs. what the 50% rule measures



*Label on the image: the **whole boll** (what your polygon covers, bracts included) vs. the **non-bract portion** (used only for the 50% visibility test). The non-bract portion is shown for example only do not annotate that seperately.*

6.2 — The baseline: a correctly annotated, fully-visible boll



This one image shows almost everything:

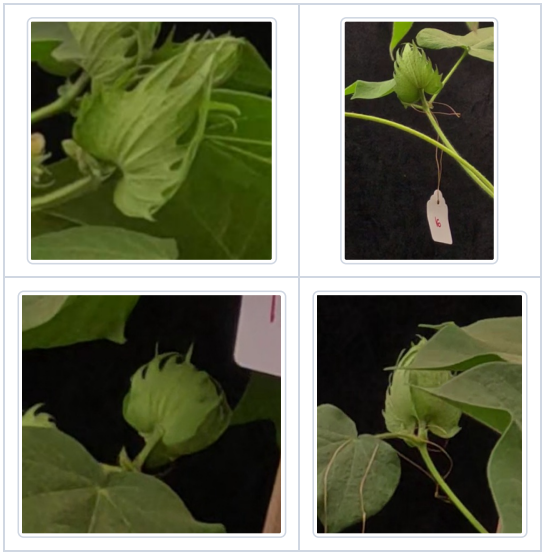
Element	What to notice
Orange polygon	The `cotton_boll` outline. It follows the **boll's actual body** — the bracts flare well past it and the polygon **does not chase them**.
Green quad	Tip region. **Straddles** the boll edge — ~2 points in, ~2 points out. Small vs. the boll, but clearly contains background.
Cyan quad	Base region. Same 2-in / 2-out rule.

`whole_visible` · `occluded_by = none` · `tip_visibility = visible` · `base_visibility = visible`

6.3 — SKIP: it's only bracts (no real boll)

This is a judgement call you will hit constantly. Sometimes what you're looking at is just the **bract structure** with no developed pod behind it, or the boll is turned so far away that you're only seeing bracts from behind.

> ### When in doubt → SKIP IT.
>
> A missing annotation costs us one training sample. A **wrong** annotation teaches the model something false.



Skip if:

- You can only see the **bracts** and no real pod behind them.
- The boll is turned so you're looking at it **from behind** and can't confirm there's a pod.
- You genuinely can't tell whether it's a boll or just bracts.
- **You'd only be sure it's a boll because you spotted a tag on it.** ← the tag is not evidence.

Do NOT skip if: you can clearly see a real pod, even if part of it is hidden. That's an *occluded boll*, not a skip — go to Step 2.

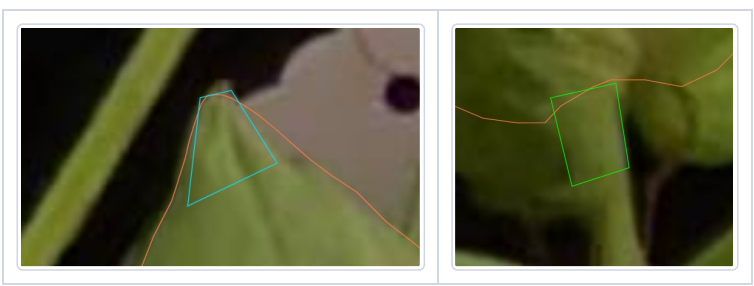
6.4 — The QUAD rule: 2 in, 2 out

CORRECT — the quad straddles the boll edge



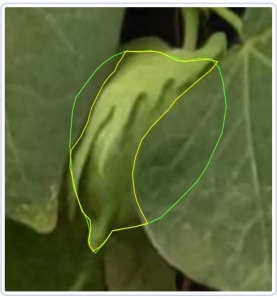
Correct — two vertices inside the boll, two outside, straddling the edge. Small compared to the boll, **but background is clearly inside the quad**.

WRONG



Wrong — all inside / all outside / too big / hugging the tip too tightly / wrong vertex count.

6.5 — Occluded by a **leaf**

	
`whole_estimate` = **pink** outline (full boll, incl. hidden part) `visible_parts` = **red** outline (only what's seen)	`whole_estimate` = **green** outline (full boll, incl. hidden part) `visible_parts` = **yellow** outline (only what's seen)

`occluded_by = leaf` · ⚠ **never** `whole_visible`

6.6 — Occluded by a **stem**



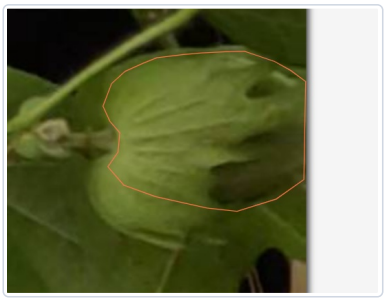
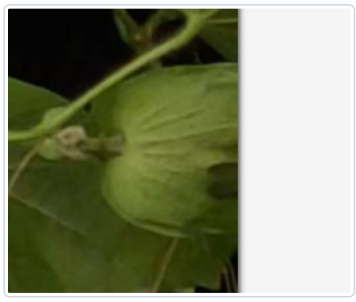
`whole_estimate` (Green) + **2** `visible_parts` (Blue & Orange) · `occluded_by = stem`

6.7 — Occluded by **another boll**



`whole_estimate` (Cyan) + `visible_parts` (Pink) · `occluded_by = other_boll`

6.8 — Cut off by the **image border (THE EXCEPTION)**

	
ANNOTATE Enough of it is in frame.	**SKIP** Too much of the **non-bract portion** is out of frame.

`whole_visible` ← **yes, whole_visible** · `occluded_by = frame_edge` **No** `whole_estimate` **.** We do not estimate off-screen geometry.

> Only when the border is the **ONLY** thing cutting it off. If a leaf also covers it → the leaf wins, go back to the normal occluded workflow.

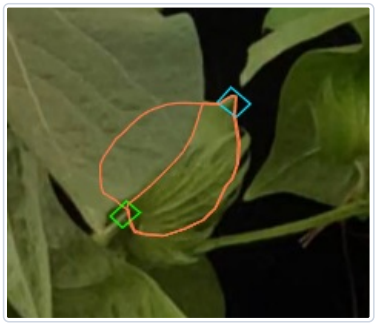
6.9 — Occluder splits the boll into two chunks



One `whole_estimate`, but **TWO** separate `visible_parts` polygons — one per visible chunk.

6.10 — SCRUB TO FIND THE TRUE SIZE (the most important technique)

Boll is occluded in some frames but is visible in previous frames.

FULLY VISIBLE	OCCLUDED
	
*Scrub back to a frame like this — now you know its **real** size.*	*Use that size for `whole_estimate` here. **Don't guess.***

Use the shape/size from frame where boll is visible as your `whole_estimate` in frame where boll is occluded. This is far more accurate than guessing.

> Only if the boll is occluded in **every single frame** do you guess the hidden part.

6.11 — Tip: guessed (draw a POINT)



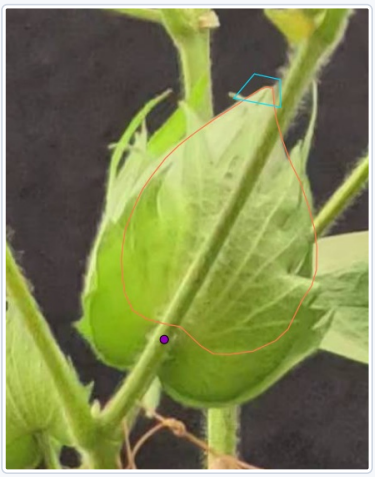
Tip is hidden, but the boll's shape tells you roughly where it is. → **POINT** (not a quad) · `tip_visibility = guessed`

6.12 — Tip: unknown (draw NOTHING)

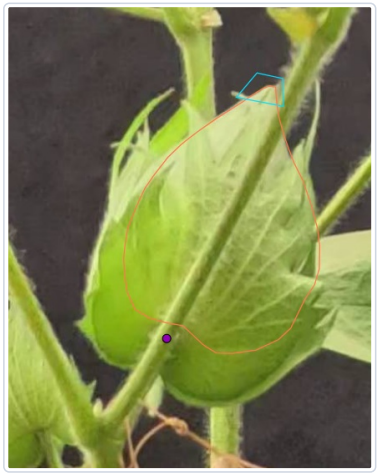


Tip can't be seen and can't be guessed (e.g. boll points away from camera, tip is out of frame, etc). → **Draw nothing** · `tip_visibility = unknown`

6.13 — Base: guessed / unknown

Base guessed (POINT)	Base unknown (NOTHING)
`right_down`, frame 0, boll_id **4**	`right_down`, frame 224, boll_id **4**
	
`base_visibility = guessed`	`base_visibility = unknown`

6.14 — Mixed: tip visible, base unknown



Tip and base are decided completely independently. It is normal for one to be a visible quad and the other to be unknown.

7. Before you submit — RUN THE VALIDATOR

Do not submit an annotation task without running this. It catches most mistakes automatically, so problems get fixed while the frames are still fresh in your head instead of weeks later.

> **I'll share `validate_annotations.py` with you** — keep it somewhere you can run it easily. You need Python installed; no other setup.

```
# 1. Export from CVAT as:  "CVAT for images 1.1"  (XML)
#    ⚠ NOT COCO — COCO format silently DELETES all your point annotations.

# 2. Run the validator
python validate_annotations.py --xml your_export.xml
```

Reading the output

```
Images checked: 300
Total issues:   0  (0 errors, 0 warnings)

ALL CLEAN — no issues found.
```

☝ **This is what you want.** If you see anything else, fix it and re-export.

Common things it will catch:

- `TIP_VIS_MISMATCH` / `BASE_VIS_MISMATCH` → your flag doesn't match what you drew
- `OCCLUSION_STRUCTURE` → occluded boll missing whole_estimate or visible_parts, or wrongly using whole_visible
- `TIP_VERTICES` / `BASE_VERTICES` → your quad doesn't have exactly 4 points
- `MISSING_BOLL_ID` → you forgot to set the boll_id
- `NO_MASK` → whole_estimate with no visible_parts
- `ORPHAN_TIP` / `ORPHAN_BASE` → a tip/base with no parent boll polygon

Fix them **in CVAT**, re-export, re-run until it says **ALL CLEAN**.

9. Quick reference card

```
┌──────────────────────────────────────────────────────────────────────────┐
│  boll_id  = THE NUMBER ON THE PHYSICAL TAG.  NEVER GUESS IT.             │
│             Can't read it? Scrub frames / check other angles.            │
│             Untagged (field data only)? → use 9000+, never reuse.        │
├──────────────────────────────────────────────────────────────────────────┤
│  Can't tell it's a boll without the tag?          →  SKIP                │
│  Can't tell boll vs. just bracts?                 →  SKIP                │
│  Less than 50% of non-bract portion visible?      →  SKIP                │
│                                                                          │
│   You always DRAW the WHOLE boll — bracts INCLUDED.                      │
│      "non-bract portion" is ONLY for the 50% visibility test.            │
│   Polygon follows the BOLL'S BODY. Bracts flare out —                    │
│      DON'T chase them. Keep the outline boll-shaped.                     │
├──────────────────────────────────────────────────────────────────────────┤
│  NOT occluded        →  whole_visible                occluded_by=none    │
│  Occluded            →  whole_estimate + visible_parts(s)                │
│                         occluded_by = leaf|stem|other_boll|other         │
│                           NEVER whole_visible on an occluded boll        │
│  Cut by border ONLY  →  whole_visible                occluded_by=frame_edge │
│                         ( the exception — no estimate)                   │
│  Border + real occluder?  → real occluder WINS → estimate + parts        │
│  Two real occluders?      → use the one covering MORE                    │
├──────────────────────────────────────────────────────────────────────────┤
│  Occluded? → SCRUB to find a frame where it IS fully visible,            │
│              and use that shape/size for whole_estimate.                 │
├──────────────────────────────────────────────────────────────────────────┤
│  TIP / BASE — the flag MUST match the drawing:                           │
│     see it clearly    →  QUAD (4 pts)  →  visibility = visible           │
│     can guess it      →  POINT         →  visibility = guessed           │
│     no idea           →  nothing       →  visibility = unknown           │
├──────────────────────────────────────────────────────────────────────────┤
│  QUAD = 4 points, ~2 INSIDE the boll, ~2 OUTSIDE.                        │
│         Small vs. the boll, but MUST include background.                 │
├──────────────────────────────────────────────────────────────────────────┤
│  Export as "CVAT for images 1.1" (XML).  NOT COCO.                       │
│  Run validate_annotations.py until it says ALL CLEAN.                    │
└──────────────────────────────────────────────────────────────────────────┘
```